Attention Mechanisms

MHA vs MQA vs GQA

Mathematical Derivations, Architecture Diagrams,
KV Cache, Memory Analysis and Interview Questions



Revision Notes and Interview Preparation

Contents

1 Foundations: Attention **3**
1.1 What is Attention? . . . 3
1.2 Scaled Dot-Product Attention . . . 3
1.3 Why divide by $\sqrt{d_h}$? . . . 4
1.4 Causal Attention Mask . . . 4

2 Multi-Head Attention (MHA) **4**
2.1 Core Idea . . . 4
2.2 MHA Architecture Diagram . . . 5
2.3 MHA Shapes . . . 5
2.4 What Does Each Head Learn? . . . 6

3 Autoregressive Decoding and KV Cache **6**
3.1 What Happens During Generation? . . . 6
3.2 KV Cache Diagram . . . 6
3.3 KV Cache Memory Formula . . . 6
3.4 Numerical Example . . . 7

4 Multi-Query Attention (MQA) **7**
4.1 Core Idea . . . 7
4.2 MQA Architecture Diagram . . . 8
4.3 MQA Shapes . . . 8
4.4 Does MQA Lose Information? . . . 8

5 Grouped-Query Attention (GQA) **9**
5.1 Core Idea . . . 9
5.2 GQA Architecture Diagram . . . 10
5.3 GQA Shapes and Memory . . . 10
5.4 Why Use GQA? . . . 11

6 MHA vs MQA vs GQA **11**
6.1 Architecture Comparison . . . 11
6.2 Visual Memory Summary . . . 11

7 Projection Parameter Counts **11**

8 Decoding Performance and Memory Bandwidth **12**
8.1 Why Does KV Cache Matter? . . . 12
8.2 Does an 8x Smaller Cache Mean 8x Faster Decoding? . . . 12
8.3 Prefill vs Decoding . . . 12

9 Common Doubts **13**
9.1 If Query Heads Are Independent, Why Share K/V? . . . 13
9.2 Are MHA's K/V Heads Identical? . . . 13
9.3 Does the Number of Query Heads Change? . . . 13
9.4 Does GQA Share the Attention Weights? . . . 13
9.5 Does KV Cache Reduce the Score Matrix? . . . 13
9.6 Can We Convert an Existing MHA Checkpoint to MQA? . . . 13
9.7 Does Sharing K/V Mean Sharing Q? . . . 13

10 Interview Questions and Answers **13**

11 Practice Problems **15**

12 Quick Revision Sheet **17**

13 Further Reading **17**

1 Foundations: Attention

1.1 What is Attention?

Attention allows a token to decide which other tokens in the sequence are relevant to its representation.
For example, in the sentence

> The animal did not cross the road because it was tired.

Attention helps the representation of "it" use relevant context from other tokens.
Three vectors are calculated:

- **Query (Q):** what information this token is looking for.
- **Key (K):** what information a token makes available for matching.
- **Value (V):** the information that is aggregated.

These are learned vector representations, not literal questions, labels or database keys.

1.2 Scaled Dot-Product Attention

Let

$$Q \in \mathbb{R}^{T_q \times d_h}, \quad K \in \mathbb{R}^{T_k \times d_h}, \quad V \in \mathbb{R}^{T_k \times d_v}.$$

First calculate the similarity scores:

$$S = QK^\top.$$

The shape is

$$(T_q \times d_h)(d_h \times T_k) = T_q \times T_k.$$

Scale the scores:

$$S_{\text{scaled}} = \frac{QK^\top}{\sqrt{d_h}}.$$

Apply softmax row-wise:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right).$$

Finally, combine the values:

$$\boxed{O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V}$$

The output shape is

$$(T_q \times T_k)(T_k \times d_v) = T_q \times d_v.$$

1.3 Why divide by $\sqrt{d_h}$?

Assume the components of Q and K have mean zero and variance one.

The dot product is

$$q^\top k = \sum_{i=1}^{d_h} q_i k_i.$$

Under the independence approximation,

$$\mathrm{Var}(q^\top k) \approx d_h.$$

Therefore, the standard deviation grows approximately as $\sqrt{d_h}$.

Dividing by $\sqrt{d_h}$ controls score magnitude and helps prevent softmax from becoming excessively saturated.

1.4 Causal Attention Mask

In autoregressive language models, token t must not use future tokens $j > t$.

Define

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Then

$$\boxed{A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)}$$

Softmax assigns zero probability to masked future positions.

For a sequence of length four, the allowed attention pattern is

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

The zeros represent forbidden attention connections.

2 Multi-Head Attention (MHA)

2.1 Core Idea

MHA uses multiple independent Query, Key and Value projections. Each attention head has its own learned projections.

Given input

$$X \in \mathbb{R}^{T \times D},$$

the projections for head i are

$$Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V.$$

The output of each head is

$$H_i = \mathrm{Attention}(Q_i, K_i, V_i).$$

Concatenate the heads and apply the output projection:

$$\boxed{\text{MHA}(X) = \text{Concat}(H_1, \ldots, H_{H_q})W^O}$$

Typically,

$$d_h = \frac{D}{H_q}.$$

Here:

- D: model hidden dimension.
- H_q: number of Query heads.
- d_h: dimension of each head.
- T: sequence length.

2.2 MHA Architecture Diagram

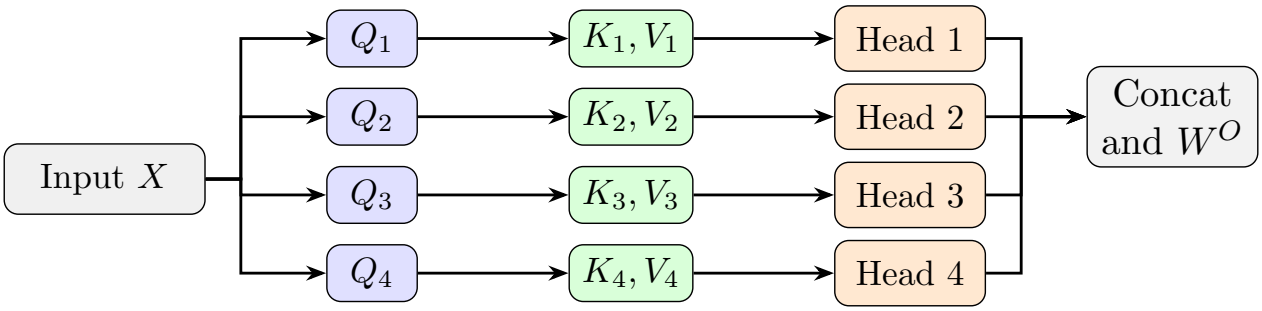


Figure 1: MHA: each Query head has its own Key and Value projections.

The diagram shows four illustrative heads. A real model can use a different number of heads.

2.3 MHA Shapes

Suppose

$$D = 512, \quad H_q = 8, \quad d_h = 64.$$

Then

$$d_h = \frac{512}{8} = 64.$$

For a sequence of length T,

$$Q, K, V \in \mathbb{R}^{T \times 8 \times 64}.$$

The total number of K/V scalar elements per token is

$$8(64) + 8(64) = 1024.$$

For one layer, the K/V cache shapes are

$$K_{\text{cache}}, V_{\text{cache}} \in \mathbb{R}^{B \times T \times 8 \times 64},$$

where B is the batch size.

2.4 What Does Each Head Learn?

Different heads can learn different relationships, such as:

- local context,
- long-distance dependencies,
- syntactic relationships,
- semantic relationships.

These are possibilities, not guarantees that every head specializes cleanly into one category.

3 Autoregressive Decoding and KV Cache

3.1 What Happens During Generation?

Suppose a model generates

$$x_1 \to x_2 \to x_3 \to x_4.$$

At the step that generates x_4, the model needs information from the previous tokens.
Without a KV cache, it may recompute K and V for all previous tokens on every decoding step.
With a KV cache, previously computed K and V vectors are stored and reused.

3.2 KV Cache Diagram

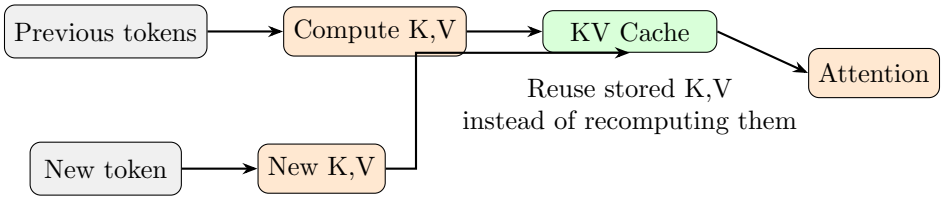


Figure 2: The KV cache stores past K/V vectors and appends new ones.

The cache avoids recomputing old K/V projections. It does not eliminate attention computation over the cached context.

3.3 KV Cache Memory Formula

For one layer, the number of stored K/V elements is

$$N_{\text{KV}} = 2BTH_{kv}d_h.$$

The factor of two is for Key and Value.
If each element occupies b bytes, memory is

$$\boxed{M_{\text{KV}} = 2BTH_{kv}d_hb}$$

Across L layers,

$$\boxed{M_{\text{total}} = 2LBTH_{kv}d_hb}$$

This formula assumes the same dimensions and precision at each layer and does not include allocator overhead or other model memory.

3.4 Numerical Example

Assume:

$$B = 1, \quad T = 4096, \quad H_{kv} = 8, \quad d_h = 64,$$

and each element uses two bytes (for example, FP16).

Then

$$M = 2(1)(4096)(8)(64)(2).$$

$$M = 8{,}388{,}608 \text{ bytes} = 8 \text{ MiB}.$$

This is the cache for one layer. For 32 identical layers,

$$M_{\text{total}} = 32(8) = 256 \text{ MiB}.$$

Reducing H_{kv} directly reduces the cache size under these assumptions.

4 Multi-Query Attention (MQA)

4.1 Core Idea

MQA keeps multiple independent Query heads but uses only one Key head and one Value head.

$$H_{kv} = 1, \qquad H_q > 1.$$

The projections are

$$Q = XW^Q,$$

$$K = XW^K, \qquad V = XW^V,$$

where the K/V projection dimensions are smaller than in standard MHA.

Each Query head computes its own attention weights against the shared K:

$$A_i = \text{softmax}\left(\frac{Q_i K^\top}{\sqrt{d_h}}\right).$$

Each head uses those weights to aggregate the shared V:

$$H_i = A_i V.$$

Thus, Query heads can still produce different outputs because their Query vectors and attention weights differ.

4.2 MQA Architecture Diagram

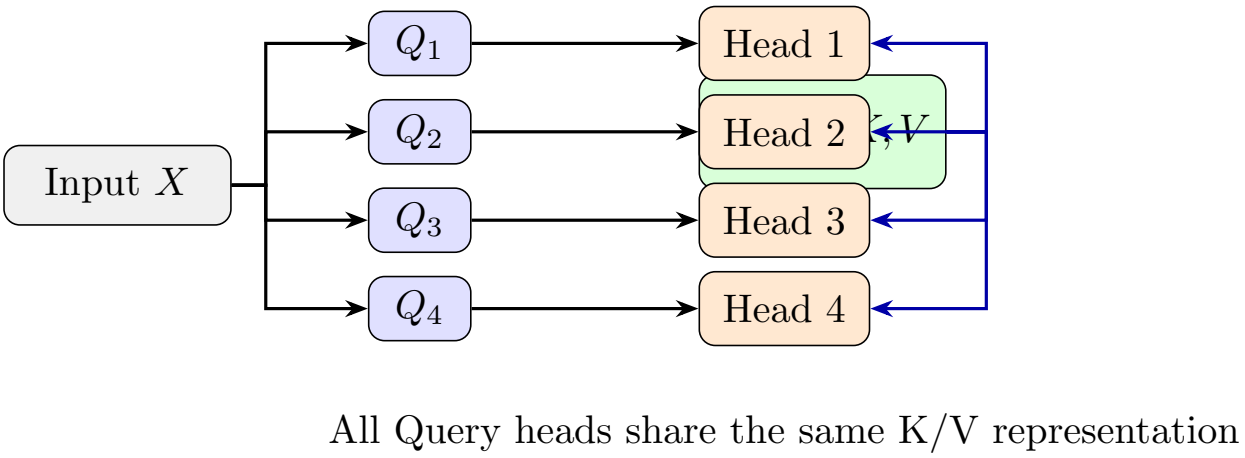


Figure 3: MQA: one shared Key head and one shared Value head.

The drawing illustrates information sharing. The exact memory access and data movement depend on the implementation.

4.3 MQA Shapes

Using

$$D = 512, \quad H_q = 8, \quad d_h = 64,$$

the Query shape remains

$$Q \in \mathbb{R}^{T\times 8\times 64}.$$

But the Key and Value shapes become

$$K, V \in \mathbb{R}^{T\times 1\times 64}.$$

K/V elements per token:

$$N_{\text{MQA}} = 2(1)(64) = 128.$$

Compare this with MHA:

$$N_{\text{MHA}} = 2(8)(64) = 1024.$$

Therefore,

$$\frac{N_{\text{MQA}}}{N_{\text{MHA}}} = \frac{128}{1024} = \frac{1}{8}.$$

MQA uses 12.5% of MHA's K/V cache, giving an 87.5% reduction under identical dimensions and precision.

4.4 Does MQA Lose Information?

MQA does not simply deduplicate identical MHA K/V vectors. In a typical MHA model, different heads have different learned K/V projections.

MQA instead learns one shared K/V representation that all Query heads use.

This creates a trade-off:

- Less KV cache memory and potentially less memory traffic.
- Less independent K/V representation capacity.
- Potential quality changes depending on the model, training procedure and task.

There is no guarantee that MQA will always have the same quality as MHA.

5 Grouped-Query Attention (GQA)

5.1 Core Idea

GQA is between MHA and MQA.

It keeps multiple Query heads but divides them into groups. Each group shares one Key head and one Value head.

$$1 < H_{kv} < H_q.$$

If Query heads are evenly divided among KV heads, group size is

$$\boxed{g = \frac{H_q}{H_{kv}}}.$$

For example,

$$H_q = 8, \qquad H_{kv} = 2,$$

so

$$g = \frac{8}{2} = 4.$$

Four Query heads share the first KV head, and the other four share the second KV head.

5.2 GQA Architecture Diagram

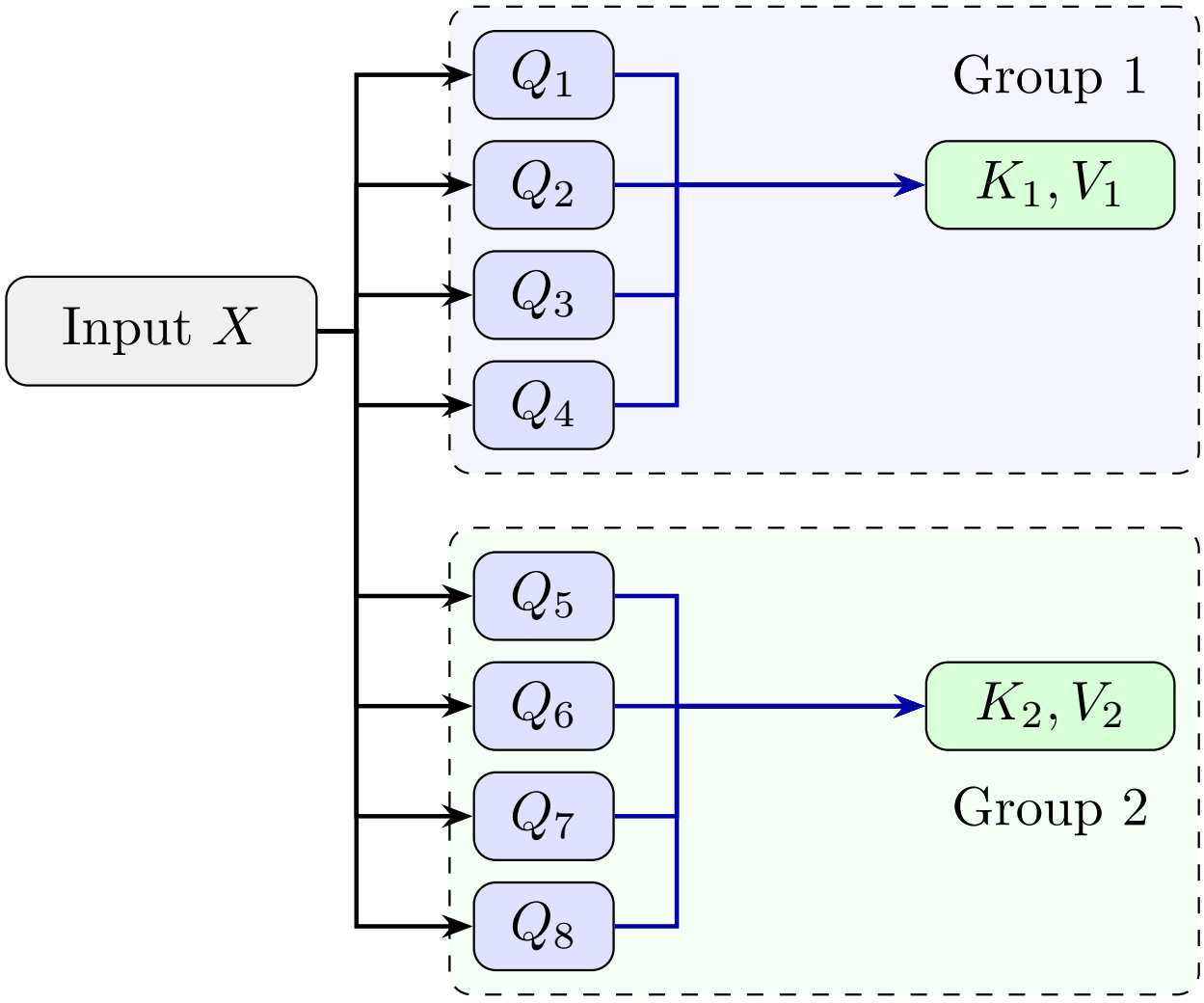


Figure 4: GQA with eight Query heads and two KV heads.

5.3 GQA Shapes and Memory

For

$$H_q = 8, \quad H_{kv} = 2, \quad d_h = 64,$$

the cache shapes are

$$Q \in \mathbb{R}^{T\times 8\times 64},$$

$$K, V \in \mathbb{R}^{T\times 2\times 64}.$$

The K/V elements per token are

$$N_{\text{GQA}} = 2(2)(64) = 256.$$

Compared with MHA:

$$\frac{N_{\text{GQA}}}{N_{\text{MHA}}} = \frac{256}{1024} = \frac{1}{4}.$$

Thus, GQA uses 25% of MHA's K/V cache, a 75% reduction under identical conditions.

5.4 Why Use GQA?

GQA aims to balance:

- the low cache cost of MQA,
- the greater K/V representation flexibility of MHA,
- practical decoding performance.

GQA is not guaranteed to be optimal for every model. Its quality and speed depend on architecture, training and hardware kernels.

6 MHA vs MQA vs GQA

6.1 Architecture Comparison



Figure 5: The primary difference is the number of KV heads.

Property	MHA	GQA	MQA
Query heads	8	8	8
KV heads	8	2	1
Queries per KV head	1	4	8
KV cache ratio	100%	25%	12.5%
KV cache reduction	0%	75%	87.5%

6.2 Visual Memory Summary

$$\underbrace{8}_{\text{MHA KV heads}} \longrightarrow \underbrace{2}_{\text{GQA KV heads}} \longrightarrow \underbrace{1}_{\text{MQA KV head}}$$

For the same batch size, sequence length, head dimension and precision, KV cache size is proportional to H_{kv}.

$$\boxed{M_{\text{KV}} \propto H_{kv}}$$

7 Projection Parameter Counts

Assume the combined Query projection has output dimension $H_q d_h$, and the combined Key and Value projections each have output dimension $H_{kv} d_h$.

Ignoring bias parameters,

$$P_Q = DH_q d_h,$$

$$P_K = DH_{kv} d_h,$$

$$P_V = DH_{kv} d_h.$$

Therefore,

$$\boxed{P_{QKV} = DH_q d_h + 2DH_{kv} d_h}$$

If $D = H_q d_h$, then

$$\boxed{P_{QKV} = D^2 + 2DH_{kv}d_h}$$

The output projection typically remains

$$W^O \in \mathbb{R}^{D\times D},$$

with

$$P_O = D^2.$$

The total attention projection parameters are consequently

$$P_{\text{attention}} = D^2 + 2DH_{kv}d_h + D^2.$$

This count excludes biases and other model components.
For standard MHA, $H_{kv} = H_q$. For MQA, $H_{kv} = 1$. For GQA, H_{kv} lies between these values.

8 Decoding Performance and Memory Bandwidth

8.1 Why Does KV Cache Matter?

During autoregressive decoding, the model generates one new token at a time. The attention operation needs access to the cached K/V vectors from previous tokens.

As the context becomes longer, the cache grows linearly:

$$M_{\text{KV}} \propto T.$$

Reducing H_{kv} can reduce the amount of cached data that needs to be stored and accessed.

8.2 Does an 8x Smaller Cache Mean 8x Faster Decoding?

No. An eightfold reduction in KV cache does not guarantee an eightfold latency improvement.

Actual speed depends on:

- GPU memory bandwidth,
- batch size and sequence length,
- attention kernel implementation,
- model compute requirements,
- cache layout and memory access patterns,
- framework and hardware optimizations.

MQA and GQA can reduce memory traffic, especially in decoding settings where reading cached K/V is a bottleneck. The actual speedup must be measured.

8.3 Prefill vs Decoding

Prefill: The model processes the input prompt, usually computing attention for many prompt tokens together.

Autoregressive decoding: The model repeatedly processes new tokens while reusing cached K/V from previous tokens.

KV sharing is particularly useful for reducing cache memory and memory traffic during decoding, although implementation and workload determine the benefit.

9 Common Doubts

9.1 If Query Heads Are Independent, Why Share K/V?

Each Query head can ask a different question of the same shared Key/Value representation.

For Query head i,

$$A_i = \text{softmax}\left(\frac{Q_i K^\top}{\sqrt{d_h}}\right), \qquad H_i = A_i V.$$

Since Q_i differs by head, the attention distribution A_i can differ too. Therefore, different heads can produce different outputs even when they share K and V.

9.2 Are MHA's K/V Heads Identical?

Generally, no. MHA learns separate K/V projections for each head. MQA does not merely store one copy of identical MHA vectors. It changes the architecture so that a shared K/V representation is learned and used by all Query heads.

9.3 Does the Number of Query Heads Change?

Not necessarily. A typical MQA or GQA configuration retains the same number of Query heads and changes the number of KV heads.

9.4 Does GQA Share the Attention Weights?

No. Query heads in a group share K and V, but each Query head computes its own attention scores and attention weights.

9.5 Does KV Cache Reduce the Score Matrix?

Not directly. The cache stores K/V projections. For a single new query token attending to T previous tokens, the score vector per Query head has length T. In standard full attention, the score matrix for a sequence of length T has shape $T \times T$ per head.

9.6 Can We Convert an Existing MHA Checkpoint to MQA?

Simply changing the KV head count is not generally enough. The projection weights and architecture no longer have matching shapes. A conversion strategy, such as combining or averaging MHA K/V heads, may provide an initialization, but additional training or fine-tuning can be needed to recover quality.

9.7 Does Sharing K/V Mean Sharing Q?

No. MQA and GQA normally retain independent Query projections for their Query heads. K/V sharing does not imply Query sharing.

10 Interview Questions and Answers

Q1. What is the main difference between MHA, MQA and GQA?

Answer:

They differ mainly in the number of Key/Value heads. MHA has a separate KV head for every Query head. MQA uses one KV head shared by all Query heads. GQA groups Query heads so that each group shares one KV head.

Q2. Why was MQA introduced?

Answer:

Autoregressive decoding can require repeatedly reading a large KV cache. MQA reduces the number of KV heads and therefore the cache size and potential memory traffic, which can improve decoding efficiency.

Q3. How much cache does GQA save when there are 8 Query heads and 2 KV heads?

Answer:

The KV cache is proportional to the number of KV heads:

$$\frac{M_{\text{GQA}}}{M_{\text{MHA}}} = \frac{2}{8} = \frac{1}{4}.$$

It uses 25% of the MHA KV cache, saving 75%.

Q4. Why not always use MQA?

Answer:

MQA minimizes the number of KV heads, but it also reduces independent K/V representation capacity. Depending on the model and task, this can affect quality. GQA provides an intermediate trade-off between cache efficiency and representation flexibility.

Q5. What is the KV cache memory formula?

Answer:

For L layers,

$$\boxed{M = 2LBTH_{kv}d_h b}$$

where B is batch size, T is cached sequence length, H_{kv} is the number of KV heads, d_h is the head dimension, and b is bytes per element.

Q6. Why is there a factor of two in the formula?

Answer:

The model stores both Key and Value vectors, so their memory contributions are added.

Q7. If K/V are shared, can Query heads still attend differently?

Answer:

Yes. Each Query head has its own Q projection and calculates its own similarity scores and softmax distribution. Sharing K/V does not force the attention weights to be identical.

Q8. Does MQA reduce the number of Query heads?

Answer:

Not necessarily. In the standard design, it reduces KV heads while retaining multiple Query heads.

Q9. Does KV caching eliminate attention computation?

Answer:

No. It avoids recomputing the past K/V projections. The new query must still compute attention against the cached context.

Q10. Why is decoding often memory-bandwidth bound?

Answer:

During decoding, the model may need to read large cached K/V tensors for each new token. In some workloads, moving these tensors from memory can be a bottleneck relative to the amount of computation performed.

Q11. Is GQA always faster than MHA?

Answer:

Not in every implementation or workload. GQA reduces KV cache size, but speed also depends on GPU kernels, hardware, batch size, sequence length and memory layout.

Q12. What is the GQA group size?

Answer:

For evenly divided groups,

$$g = \frac{H_q}{H_{kv}}.$$

For 8 Query heads and 2 KV heads, each KV head is shared by 4 Query heads.

11 Practice Problems

Problem 1: Calculate Head Dimension

A model has $D = 768$ and $H_q = 12$.

Calculate d_h.

Solution:

$$d_h = \frac{768}{12} = 64.$$

Problem 2: Calculate GQA Cache Reduction

A model has 16 Query heads and 4 KV heads.

Calculate:

(a) group size,

(b) KV cache ratio relative to MHA,

(c) percentage reduction.

Solution:

$$g = \frac{16}{4} = 4.$$

$$\text{Cache ratio} = \frac{4}{16} = 25\%.$$

$$\text{Reduction} = 100\% - 25\% = 75\%.$$

Problem 3: Calculate KV Cache Memory

Given

$$L = 24, \quad B = 2, \quad T = 2048, \quad H_{kv} = 4, \quad d_h = 64, \quad b = 2,$$

calculate total cache memory.

Solution:

$$M = 2LBTH_{kv}d_hb.$$

$$M = 2(24)(2)(2048)(4)(64)(2).$$

$$M = 402{,}653{,}184 \text{ bytes}.$$

Using binary units,

$$\boxed{M = 384 \text{ MiB}}.$$

Problem 4: Compare MHA and MQA

A model has 8 Query heads, head dimension 64, and FP16 cache values.

Calculate K/V elements per token for MHA and MQA.

Solution:

MHA:

$$2(8)(64) = 1024 \text{ elements}.$$

MQA:

$$2(1)(64) = 128 \text{ elements}.$$

MQA uses one-eighth as many K/V elements.

Problem 5: Explain the Trade-off

Why might a model choose GQA instead of MHA or MQA?

Solution:

GQA can reduce KV cache memory and memory traffic substantially while retaining multiple KV representations. It offers a middle ground, although the best configuration depends on quality and performance measurements.

12 Quick Revision Sheet

Concept	Remember
Attention	$\text{softmax}(QK^\top/\sqrt{d_h})V$
MHA	Independent Q, K and V projections per head
MQA	Many Query heads share one KV head
GQA	Groups of Query heads share KV heads
Group size	$g = H_q/H_{kv}$, for evenly divided groups
KV cache	Stores past Key and Value vectors
Cache formula	$2LBTH_{kv}d_hb$ bytes
MHA cache ratio	100% baseline
GQA cache ratio	H_{kv}/H_q of MHA, with equal head dimensions
MQA cache ratio	$1/H_q$ of MHA, with equal head dimensions
Performance	Cache reduction can help decoding, but speedup is hardware- and workload-dependent

13 Further Reading

1. Shazeer, N. (2019). *Fast Transformer Decoding: One Write-Head is All You Need.* https://arxiv.org/abs/1911.02150

2. Ainslie, J. et al. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.* https://arxiv.org/abs/2305.13245

3. Vaswani, A. et al. (2017). *Attention Is All You Need.* https://arxiv.org/abs/1706.03762

End of Revision Notes